

1. INTRODUCTION

The volume of digital commercial contracts has grown rapidly as organisations increasingly negotiate, sign, and store agreements electronically. Yet the clauses that matter most within these contracts — limitation of liability, termination rights, exclusivity, governing law — remain difficult to locate manually. A single contract may run to dozens of clauses spread across many pages, and a reviewer must read the entire document to be confident that no material clause has been missed. As transaction volume grows, this manual process does not scale, and important clauses are sometimes overlooked simply because a reviewer runs out of time.

Natural Language Processing (NLP) and Machine Learning (ML) offer a practical way to reduce this burden by automatically identifying and categorising contract clauses. ClauseIQ is a system that extracts clauses from legal contracts, classifies them into predefined categories drawn from the Contract Understanding Atticus Dataset (CUAD), and provides an intelligent search capability so that a user can retrieve clauses matching a natural-language query. The project deliberately uses classical, well-understood ML techniques — TF-IDF vectorisation combined with Multinomial Naive Bayes, Linear SVM, Logistic Regression and Random Forest — rather than deep learning or transformer-based architectures, in keeping with the scope of a semester mini project.

CUAD provides 510 real commercial contracts, expert-annotated across 41 clause categories, and forms the training and evaluation dataset for ClauseIQ’s classifier. Clause text is represented numerically using Term Frequency–Inverse Document Frequency (TF-IDF), which converts each clause into a sparse vector that highlights category-discriminating vocabulary while remaining interpretable and inexpensive to compute. The same TF-IDF representation supports ClauseIQ’s second capability: intelligent search, in which a free-text query is compared against the corpus of clauses using cosine similarity, returning the most relevant matches without requiring an exact keyword match.

Beyond the machine-learning pipeline, ClauseIQ is delivered as a working application. A FastAPI backend performs PDF text extraction, clause splitting, text preprocessing, TF-IDF vectorisation, ML classification and cosine-similarity search, while a React frontend allows a user to upload a contract, view classification results, and search across clauses. Extracted contract and clause data, predicted categories, and classification confidence scores are persisted in a Supabase PostgreSQL database, which stores contract metadata and clause-level records in a one-to-many relationship. This report documents the dataset analysis, preprocessing design, algorithm selection, application architecture, database design, and system design (use case, activity, sequence and class diagrams) that together define the current scope of ClauseIQ.

2. SUPPORTING LITERATURE

2.1. Literature Review

The surge in legal-text production has increased the workload of legal professionals, and the specialised, dense language of legal documents makes many legal-NLP tasks repetitive and time-consuming to perform manually. Although Legal NLP has advanced steadily since 2015 with the growth of Deep Learning and, more recently, Large Language Models, no systematic account of this progress existed up to 2022. This paper addresses that gap through a Systematic Mapping Study (SMS): a structured literature search across four reputable academic indexers that began with 536 candidate papers and, after applying explicit inclusion/exclusion criteria, filtered them down to 75 pivotal articles. For ClauseIQ, this systematic mapping study is valuable because it confirms that Multiclass Classification of legal text — precisely the task ClauseIQ performs across CUAD’s 41 clause categories — is one of the most actively researched problem families in Legal NLP, and because it spans both classical ML and deep-learning solutions without concluding that one dominates, supporting a TF-IDF plus classical-ML pipeline as a literature-grounded starting point rather than an outdated shortcut.

Paper 1

Authors	Quevedo E., Cerny T., Rodriguez A., Rivas P., Yero J., Sooksatra K., Zhakubayev A., Taibi D.
Year	2024 (IEEE Access, vol. 12, pp. 145286–145317, DOI: 10.1109/ACCESS.2023.3333946)
Title	Legal Natural Language Processing From 2015 to 2022: A Comprehensive Systematic Mapping Study of Advances and Applications
Main Approach	Systematic Mapping Study (SMS): structured search across four academic indexers, screening 536 candidate papers down to 75 pivotal articles, then categorising them by research problem.
Dataset / Domain	Literature corpus (not a single experimental dataset); domain is Legal NLP research published 2015–2022.
Key Findings	Legal NLP concentrates on a small number of dominant task families — Multiclass Classification, Summarization, Question Answering — addressed by both classical, feature-based ML and deep-learning methods; identifies recurring gaps in dataset curation, legal ontologies and data accessibility.
Relevance to ClauseIQ	Confirms multiclass legal-text classification (ClauseIQ’s core task across CUAD’s 41 categories) as a core, well-studied problem, and legitimises a classical TF-IDF + ML pipeline as a literature-grounded design choice rather than an outdated shortcut.

Table 2.1: Summary of Paper 1

Evaluation Metrics and Results (Paper 1). Quevedo et al. (2024) is a Systematic Mapping Study and does not train or evaluate a model of its own — it reports no original Accuracy, Precision, Recall or F1-score. The scores below were reported by the *primary studies reviewed* within the paper, not by Quevedo et al. themselves:

Model (reviewed primary study)	Metric	Score
CNN + Word2Vec	Accuracy	72.4% (15 general categories)
LSTM	Accuracy	Up to 90%
Ensemble of BERT models	Accuracy	0.7037
BERT-based classifier	F-score	0.70
RoBERTa-base (vs. SVM)	F1 / MRR	31% F1 improvement; 41% MRR improvement
Custom LegalBERT (Over-ruling task)	F1-score	0.973
Custom LegalBERT (Terms of Service task)	F1-score	0.812
BigBird	Weighted-F1	0.809

This is a systematic literature review (61 selected publications, 2017–2023) that maps how Transformer/LLM-based NLP has been applied across the legal domain, including legal document analysis, case prediction, legal search, regulatory compliance and — most relevant to ClauseIQ — contract review. The survey traces a clear progression from classical, feature-based text classification (bag-of-words / TF-IDF with SVM or Naive Bayes) toward BERT-family and, more recently, LLM-based approaches, and explicitly discusses CUAD as one of the standard benchmark datasets used for contract clause identification. For ClauseIQ, the survey confirms that clause-level classification and semantic clause search are recognised, well-studied sub-tasks within Legal NLP, and documents the accuracy/interpretability/cost trade-off that supports the project’s decision to begin with a classical ML baseline before considering deep-learning or LLM-based extensions.

Paper 2

Authors Siino M., Falco M., Croce D., Rosso P.
 Year 2025 (IEEE Access, vol. 13, pp. 18253–18276, DOI: 10.1109/ACCESS.2025.3533217)
 Title Exploring LLMs Applications in Law: A Literature Review on Current Legal NLP Approaches

Main Approach	PRISMA-style systematic literature review of 61 publications (2017– 2023), selected via database search and backward snowballing, assessed with a CASP-based quality checklist.
Dataset / Domain	Legal NLP applications across contract review, legal document analysis, case prediction, legal search and regulatory compliance; explicitly discusses CUAD as a benchmark for contract-clause identification.
Key Findings	Traces a progression from classical TF-IDF/SVM/Naive-Bayes pipelines toward BERT-family and LLM-based approaches, and documents a persistent trade-off between the low cost/high interpretability of classical ML and the higher (but more expensive, less transparent) accuracy of transformer-based models.
Relevance to ClauseIQ	Directly names CUAD as a standard contract-clause benchmark, validating ClauseIQ’s dataset choice, and documents the classical-ML-vs-LLM trade-off that justifies starting with TF-IDF and classical classifiers rather than a transformer/LLM pipeline.

Table 2.2: Summary of Paper 2

Evaluation Metrics and Results (Paper 2). Siino et al. (2025) is a literature review and does not train or evaluate a model of its own — it reports no original Accuracy, Precision, Recall or F1-score. The scores below are *cited from studies included in the review*, not produced by Siino et al. themselves:

Model / approach (re-viewed study)	Metric	Score
XLNet/BiGRU	Accuracy	78%
Human domain expert (same task)	Accuracy	94%
Custom legal-vocabulary domain pretraining	F1 gain	7.2% F1 gain (12% improvement over BERT)
BERT-based model (positive outcome prediction)	F1-score	75.06
ITALIAN-LEGAL-BERT vs. ITALIAN-BERT	Improvement	18.2% (civil); 15.4% (criminal)
XLM-R baseline	Aggregated benchmark score	61.3

Text preprocessing is a critical step in any NLP pipeline, and named entities — person names, organisations, addresses, dates — pose a particular problem: removing them discards potentially useful structure, while leaving them as raw tokens adds noise and can leak identifying information. This paper proposes a model-agnostic alternative: using a spaCy Named Entity Recognition (NER) tagger to detect named entities and replace each one with a generic placeholder for its entity type (e.g., PERSON, ORG, GPE) rather than deleting it or leaving it verbatim. This paper is the most directly applicable of the three to ClauseIQ’s own preprocessing design, since it operates on legal contract paragraphs using the same broad workflow ClauseIQ follows: clean text, vectorise it, and classify it with a standard classifier. CUAD clauses, like the LPA paragraphs studied here, are dense with party names, company names, dates and addresses that carry little category-discriminating signal but do carry privacy and dimensionality costs if left untreated.

Paper 3

Authors Kutbi M.
 Year 2023 (IEEE Access, vol. 11, DOI: 10.1109/ACCESS.2023.3325895)
 Title Named Entity Recognition Utilized to Enhance Text Classification While Preserving Privacy

Main Approach	Uses a spaCy Named Entity Recognition (NER) tagger to detect named entities and replace each with a generic entity-type placeholder (e.g., PERSON, ORG, GPE) prior to text classification, instead of deleting or retaining raw entity text.
Dataset / Domain	An in-house legal dataset of Limited Partnership Agreement (LPA) contract paragraphs (20 classes) plus two public non-legal datasets, used to test cross-domain generality.
Key Findings	Entity-type replacement improved downstream text-classification accuracy and reduced feature-vector dimensionality relative to leaving raw entities in place, across both the legal and non-legal datasets, while anonymising sensitive identifying information.
Relevance to ClauseIQ	Operates on legal contract paragraphs using the same broad TF-IDF-classification workflow ClauseIQ follows; motivates a future refinement to ClauseIQ’s preprocessing that generalises party names, organisation names and dates in CUAD clauses rather than leaving them as noisy tokens.

Table 2.3: Summary of Paper 3

Evaluation Metrics and Results (Paper 3). Unlike Papers 1 and 2, Kutbi (2023) is an experimental study and reports *original* Linear SVM accuracy results using 5-fold cross-validation repeated 10 times. Precision, Recall and F1-score are **Not Reported**.

Dataset	Config.	Linear SVM	Linear SVM + NER Replacement
Legal LPA	1-gram	55.37%	58.31%
Legal LPA	2-gram	55.60%	63.76%
Legal LPA	3-gram	57.48%	62.83%
Movie Reviews	1-gram	81.00%	81.90%
Movie Reviews	2-gram	82.50%	82.80%
Movie Reviews	3-gram	82.60%	82.50%
Comparative tences	Sen- 1-gram	68.30%	68.54%
Comparative tences	Sen- 2-gram	66.66%	67.37%
Comparative tences	Sen- 3-gram	65.25%	66.59%

Most relevant to ClauseIQ: on the legal LPA dataset, Linear SVM with Named Entity Replacement achieved its best accuracy of **63.76%** using 2-grams — the strongest result across all configurations tested on legal text.

2.2. Literature Review Summary

Paper	Approach	Key Finding	Relevance to ClauseIQ
Quevedo et al. (2024)	Systematic Mapping Study of Legal NLP (2015–2022)	Multiclass classification is a dominant, established Legal NLP task	Legitimises classical TF-IDF baseline for CUAD clause classification
Siino et al. (2025)	PRISMA-style review of LLM/NLP in law (2017–2023)	CUAD named as a standard contract-clause benchmark; classical-vs-LLM cost/accuracy trade-off documented	Validates CUAD choice; supports classical-ML-first strategy
Kutbi (2023)	NER-based entity-type replacement before text classification	Improves accuracy, reduces dimensionality, preserves privacy on a legal contract dataset	Motivates future NER-based cleaning of party names/dates in CUAD clauses

Table 2.4: Overall Summary of All Papers

2.3. Findings and Proposals

The three reviewed papers together confirm that multiclass legal-text classification is an established, actively researched Legal NLP problem (Quevedo et al., 2024), that CUAD is a literature-recognised benchmark specifically for contract-clause identification (Siino et al., 2025), and that classical, TF-IDF-based pipelines remain a documented, defensible choice relative to deep-learning and LLM-based alternatives, particularly for a scope-controlled mini project. Kutbi (2023) additionally shows, on a legal-contract classification task using the same broad workflow ClauseIQ follows, that lightweight preprocessing refinements such as NER-based entity generalisation can improve classical-classifier accuracy without introducing deep architectures.

The research gap these papers leave open is practical rather than theoretical: none of the three papers implements an end-to-end, deployable clause classification and search application built specifically on CUAD with a persistent database and a usable interface. Based on the literature findings above, ClauseIQ proposes: (1) CUAD-based clause classification across all 41 annotated categories; (2) TF-IDF as an interpretable, low-cost feature representation; (3) four classical classifiers — Multinomial Naive Bayes, Linear SVM, Logistic Regression and Random Forest — trained and compared on this representation; (4) cosine similarity over the same TF-IDF space for intelligent clause search; (5) a contract-level train/test split so that clauses from the same contract do not leak across training and evaluation; and (6) a working application layer — a FastAPI backend performing extraction, preprocessing, classification and search, a React frontend for user interaction, and a Supabase PostgreSQL database for persisting contract and clause records — that operationalises the classical ML pipeline rather than leaving it as an offline experiment.

3. SYSTEM ANALYSIS

This chapter documents dataset analysis, preprocessing, visualisation, algorithm study, project pipeline, feasibility, system environment and database design for ClauseIQ. All statistics in this chapter are drawn directly from the project’s Exploratory Data Analysis (EDA) report and are not estimated.

3.1. Analysis of the Dataset

3.1.1. About the Dataset

ClauseIQ is built on the Contract Understanding Atticus Dataset (CUAD), a corpus of 510 real commercial contracts sourced from the U.S. Securities and Exchange Commission’s EDGAR filing system and expert-annotated across 41 clause categories (e.g., Governing Law, Anti-Assignment, Cap On Liability, Non-Compete). CUAD’s raw annotations are contract-level: each contract is paired with the text spans that lawyers identified for each of the 41 categories. For ClauseIQ’s classification and search use case, these contract-level annotations were flattened into a clause-level dataset, where each row represents one annotated clause together with its source contract and category label.

The resulting ClauseIQ dataset contains 12,204 rows and 3 columns:

- `contract_id` — identifies the source contract (510 unique contracts)
- `clause_text` — the extracted clause text
- `category` — the ground-truth clause category (41 unique categories)

This three-column, clause-level structure is the working dataset used throughout the remainder of this chapter and is what ClauseIQ’s TF-IDF and classification pipeline operates on.

3.1.2. Explore the Dataset

a) Shape, schema and missing values

```
Shape: (12204, 3)
Columns: contract_id (object), clause_text (object), category (object)
Missing values:
contract_id    0
clause_text    0
category       0
```

Figure 3.1: Dataset shape, schema and missing-value check

All three columns are complete, with zero missing values — confirmed both in the tabular check above and in the EDA’s missing-value chart (0.0% nulls for every column).

b) Duplicate analysis

```
Exact duplicate rows:      0
Duplicate clause texts:    1,655 (13.56%)
Unique distinct clause texts: 10,549
```

Figure 3.2: Duplicate-row and duplicate-text check

There are no exact duplicate rows, but 1,655 clause texts repeat across different contracts. This is expected: many CUAD contracts share boilerplate provisions (e.g., identical “prior written consent” clauses) drawn from similar templates, rather than indicating a data-quality problem.

c) Contract-level distribution

Mean clauses per contract: 23.93
 Median clauses per contract: 19.00
 Minimum clauses per contract: 1
 Maximum clauses per contract: 93

Figure 3.3: Clauses-per-contract summary statistics



Figure 3.4: Distribution of clauses per contract (N = 510 contracts)

d) Class distribution

The 41 categories are strongly imbalanced. The largest class, *Parties*, has 1,251 clauses, while the smallest, *Price Restrictions*, has only 27 — an imbalance ratio of 46.33 : 1. Section 3.2.3 discusses the modelling implications of this imbalance in detail.

e) Word count, character length and sentence count

	Word Count	Character Length	Sentence Count
Minimum	3	9	1
Maximum	479	3,169	24
Mean	45.89	289.16	1.63
Median	36.00	224.00	1.00
Std. Dev.	43.66	277.38	1.25

Figure 3.5: Word-count, character-length and sentence-count statistics

Using the interquartile-range (IQR) rule, clauses longer than 127 words (6.20% of the dataset) or 807.5 characters (6.22%) are statistically classified as outliers. The EDA report’s qualitative review of these cases found that the shortest clauses (3 words) are valid CUAD annotations such as contract titles and party names (e.g., “TRADEMARK LICENSE AGREEMENT”), while the longest clauses (380–479 words) are fully articulated, binding provisions such as detailed Non-Compete or Source Code Escrow terms. Neither extreme is a data-quality artefact.

f) Frequent terms and bigrams

The five most frequent content words (stopwords removed) are agreement (7,686), shall (7,352), party (4,685), term (2,525) and section (2,300). The five most frequent bigrams are agreement shall (1,027), set forth (957), effective date (821), prior written (760) and written notice (753). These are examined further, with supporting figures, in Section 3.3.

3.2. Data Preprocessing

3.2.1. Data Cleaning

ClauseIQ’s preprocessing pipeline is applied to `clause_text` as follows:

1. Text extraction — extracting raw text from an uploaded PDF contract.
2. Clause splitting — dividing the extracted contract text into individual clause-level spans.
3. Lowercase conversion — normalising case so that “Agreement” and “agreement” are treated identically.
4. Punctuation / special-character handling — removing characters that do not carry legal-text meaning while retaining structurally relevant punctuation.
5. Whitespace normalisation — collapsing repeated spaces, tabs and newlines into single spaces.
6. Tokenization — splitting cleaned text into individual word tokens for vectorisation.
7. Stopword handling — removing common English stopwords that carry little category-discriminating signal.

8. TF-IDF vectorization — converting cleaned, tokenised clause text into numerical feature vectors (Section 3.4.1).
9. N-gram handling — including bigrams alongside unigrams, since collocations such as “prior written” and “governing law” carry meaning beyond individual words.

The EDA’s short-clause audit shows that very short spans such as Parties and Document Name entries (many under 5 words, e.g., party names or document titles) are valid, correctly annotated CUAD categories rather than noise: removing clauses under 5 words would eliminate 89.8% of all Parties records and 55.0% of all Document Name records. These short spans are therefore retained, not automatically filtered out. Long clauses (up to 479 words) are likewise retained, since they represent fully articulated, binding provisions (e.g., detailed Non-Compete terms) whose operative conditions would be lost if truncated; sublinear TF scaling is used during TF-IDF vectorisation (Section 3.4.1) to prevent these longer clauses from dominating the feature space.

3.2.2. Analysis of Feature Variables

The main textual feature used by ClauseIQ is `clause_text`. Three derived text metrics, computed directly from `clause_text`, are used during EDA to characterise the corpus (they are not passed to the classifiers as separate numeric features, since TF-IDF operates on the text itself):

- `word_count` — number of words in a clause (mean 45.89, median 36, maximum 479). Longer clauses tend to be substantive covenants, while very short ones are typically titles or party names.
- `char_len` — number of characters in a clause (mean 289.16, median 224, maximum 3,169), used as a cross-check on word count.
- `sentence_count` — number of sentences in a clause (mean 1.63, median 1), showing that most clauses are single-sentence provisions.

`char_len` and `word_count` are almost perfectly correlated (Pearson correlation = 0.9945), confirmed in Figure 3.9 (Section 3.3): longer clauses are consistently both longer in words and in characters, as expected, so the two metrics carry largely redundant length information for modelling purposes.

3.2.3. Analysis of Class Variables

The target (class) variable for ClauseIQ’s classifier is `category`, with 41 possible classes. `category` is the variable being predicted and is not itself a feature variable.

As Section 3.1.2(d) shows, the class distribution is highly imbalanced: Parties (1,251 clauses) is the largest class and Price Restrictions (27 clauses) is the smallest, an imbalance ratio of 46.33 : 1. Under this imbalance, overall accuracy is an uninformative metric, since a classifier that always predicts Parties would score deceptively well while failing on every minority category. Appropriate evaluation therefore requires:

- Macro-averaged F1-score — the primary metric, weighting all 41 categories equally regardless of size.
- Weighted-averaged F1-score — a secondary metric reflecting overall clause volume.

- Per-class precision, recall and F1-score — to check that rare categories (e.g., Price Restrictions) are not being ignored.
- Confusion matrix — to identify which categories are commonly confused with one another.

Class weighting (`class_weight='balanced'` in scikit-learn's SVM and Logistic Regression implementations) and Stratified K-Fold cross-validation are proposed techniques for handling this imbalance during model training; they are recommended by the project's EDA analysis but are not yet reflected in reported results, since model training and evaluation are addressed in a later implementation stage. Evaluation results will be reported after model training.

Once the stratified split is generated at the modelling stage, three validation checks will be carried out before proceeding to model training. First, the number of unique contracts represented in each split will be verified, to confirm that clauses from the same contract have not been divided across both the training and test sets. Second, the label (category) distribution within each split will be verified to confirm that stratification has preserved proportional representation of every category. Third, it will be verified that no clause category is missing from either the training or the test data, since a category absent from the test set could not be evaluated and one absent from the training set could not be learned. These checks help ensure that the dataset split is representative and suitable for reliable model training and evaluation.

3.3. Data Visualization

Six figures from the project's EDA output were selected as the most informative for this report; additional EDA figures (e.g., boxplots and the missing-value chart) are omitted here since their findings are already summarised in Section 3.1.2 and Section 3.2.2.

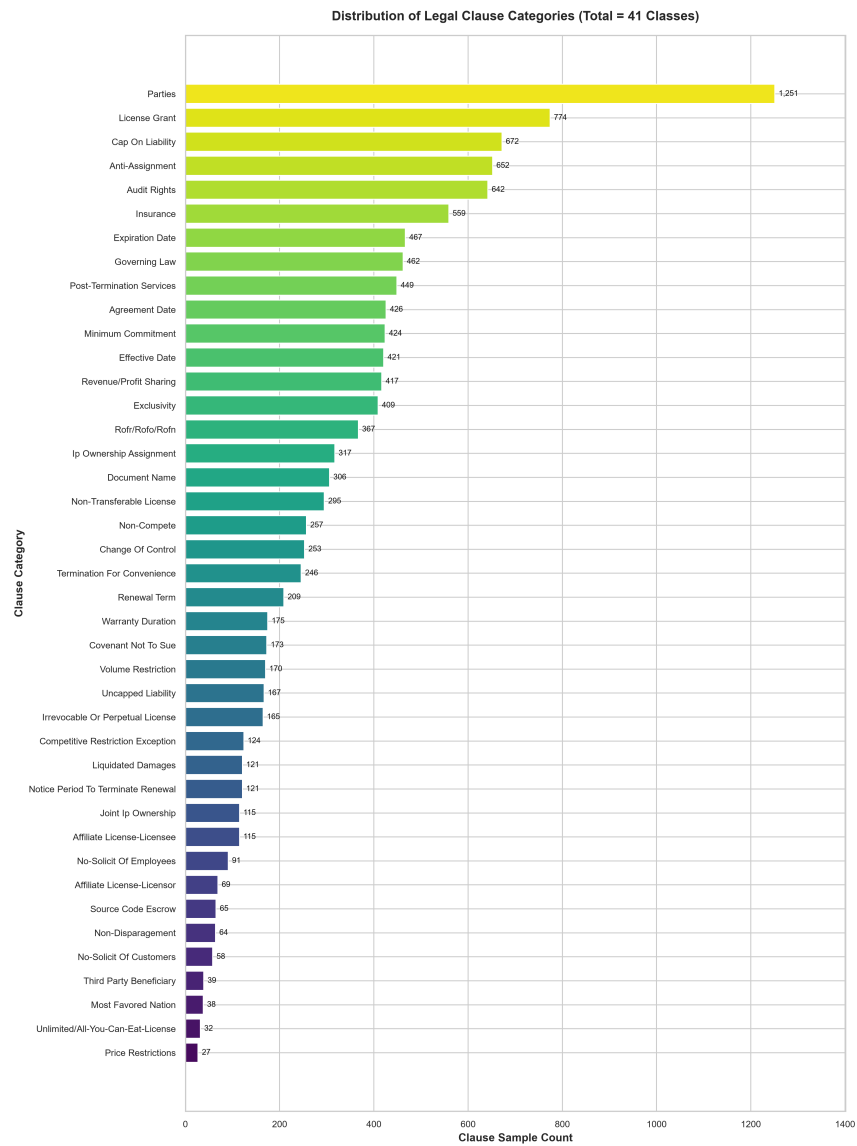


Figure 3.6: Distribution of legal clause categories in the ClauseIQ dataset (all 41 categories)

Figure 3.6 shows the full 41-category distribution: a small number of categories (Parties, License Grant, Cap On Liability, Anti-Assignment, Audit Rights) account for a disproportionate share of clauses, while many categories — particularly licensing-related classes such as Affiliate License-Licensor and Price Restrictions — have fewer than 100 examples. This confirms the 46.33:1 imbalance ratio discussed in Section 3.2.3.

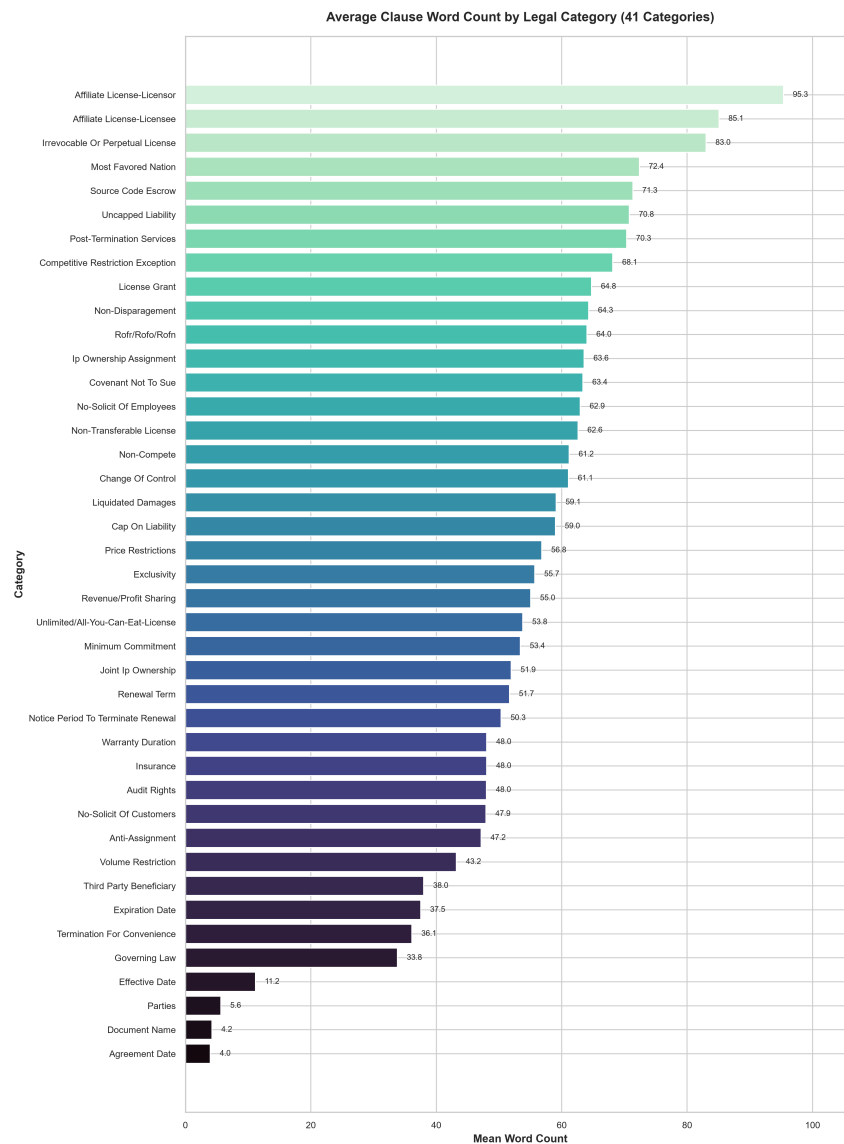


Figure 3.7: Average clause word count by legal category

Figure 3.7 shows that clause length varies systematically by category: metadata-style categories such as Agreement Date (mean 4.0 words) and Parties (mean 5.6 words) are extremely short, while substantive covenants such as Affiliate License-Licensor (mean 95.3 words) and Irrevocable Or Perpetual License (mean 83.0 words) are far longer. This variation is a further reason category should not be inferred from length alone, and supports TF-IDF’s term-based (rather than length-based) representation.

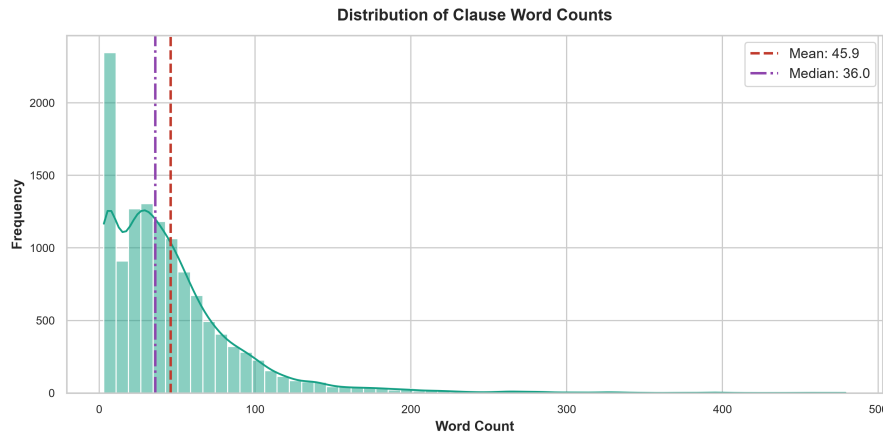


Figure 3.8: Distribution of clause word counts

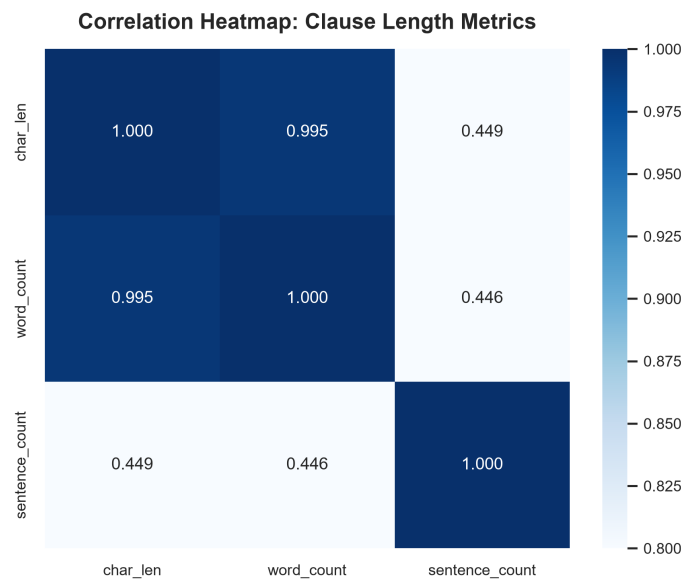


Figure 3.9: Correlation heatmap of clause length metrics

Figure 3.9 confirms the near-perfect correlation (0.9945) between `char_len` and `word_count` discussed in Section 3.2.2, and a weaker correlation (approximately 0.45) between `sentence_count` and the two length metrics, indicating that longer clauses are not simply proportionally longer sentences but often contain more numerous, shorter sentences.

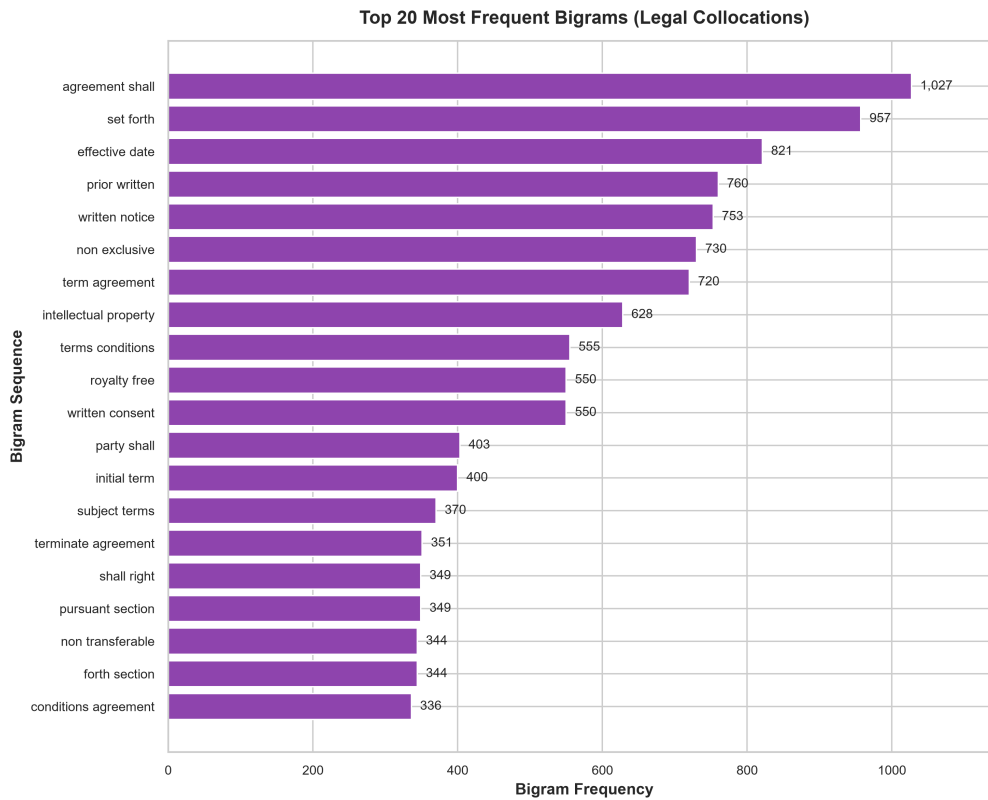


Figure 3.10: Top 20 most frequent bigrams (legal collocations) in the clause corpus

Figure 3.10 shows that the most frequent bigrams — “agreement shall”, “set forth”, “effective date”, “prior written”, “written notice” — are standard contract boilerplate rather than category-specific vocabulary, reinforcing the recommendation (Section 3.4.1) to include bigrams (`ngram_range=(1, 2)`) in TF-IDF so that more specific collocations can still contribute discriminative signal alongside these common phrases.

3.4. Analysis of Architecture and/or Algorithms

ClauseIQ’s overall architecture follows a layered design:

React Frontend → *FastAPI Backend* → *ML/NLP Processing* → *Supabase PostgreSQL*

The React frontend collects user input (contract upload or search query) and displays results. The FastAPI backend performs PDF text extraction, clause splitting, preprocessing, TF-IDF vectorisation, ML classification and cosine-similarity search. Supabase PostgreSQL persists contract and clause records (Section 3.8).

3.4.1. Detailed Study Report of Proposed Algorithms

A. Multinomial Naive Bayes

A probabilistic classifier that assumes feature (term) independence given the class, estimating each category’s likelihood from term frequencies. It is well suited to text classification because it handles sparse, high-dimensional TF-IDF matrices efficiently and requires very little training data per class. *Advantages*: fast to train, simple, a strong baseline for multi-class text

problems. *Limitations*: the independence assumption is unrealistic for correlated legal phrasing, which can limit accuracy on categories with overlapping vocabulary. *Role in ClauseIQ*: a fast, interpretable baseline classifier against which the other three models are compared.

B. Linear SVM

Support Vector Machines find the maximum-margin hyperplane separating classes. A linear kernel is well suited to high-dimensional, sparse TF-IDF data, where the number of features (thousands of terms) is much larger than what many other classifiers handle efficiently, and a one-vs-rest scheme extends the binary SVM to ClauseIQ’s 41-class problem. *Advantages*: strong empirical performance on text classification, robust to high dimensionality. *Limitations*: less naturally probabilistic than Logistic Regression; sensitive to the choice of regularisation parameter. *Role in ClauseIQ*: the primary candidate for high-accuracy clause classification.

C. Logistic Regression

A linear probabilistic classifier trained with multinomial (softmax) loss, producing class probabilities rather than only a hard label. *Advantages*: interpretable per-class term coefficients, low computational cost, well-calibrated probabilities useful for ranking candidate categories. *Limitations*: assumes a linear decision boundary in feature space, which may underfit complex category overlaps. *Role in ClauseIQ*: a probabilistic classifier whose confidence scores can support the `confidence_score` field stored per clause (Section 3.8).

D. Random Forest

An ensemble of decision trees trained on bootstrap samples and random feature subsets, aggregated by majority vote. *Advantages*: captures non-linear feature interactions, robust to noisy or irrelevant terms. *Limitations*: higher memory use on very high-dimensional sparse TF-IDF input; less interpretable than linear models. *Role in ClauseIQ*: a non-linear classifier included for comparison against the three linear/probabilistic baselines above.

E. TF-IDF

Term Frequency–Inverse Document Frequency transforms clause text into numerical feature vectors. Term Frequency (TF) counts how often a term appears in a clause; Inverse Document Frequency (IDF) down-weights terms that appear in many clauses across the corpus (e.g., “shall”, “agreement”) and up-weights terms that are comparatively rare and therefore more category-discriminating. The EDA analysis recommends `sublinear_tf=True` (to dampen the influence of very long clauses), `max_df=0.85` (to suppress ubiquitous non-discriminative terms) and `min_df=3` (to prune rare noise), with `ngram_range=(1,2)` to capture legal collocations.

F. Cosine Similarity

For intelligent search, both the clause corpus and an incoming query are projected into the same TF-IDF vector space. Cosine similarity measures the angle between the query vector and each clause vector; a smaller angle (cosine value closer to 1) indicates greater similarity. Clauses are ranked by this score and the highest-similarity clauses are returned as search results, without requiring an exact keyword match.

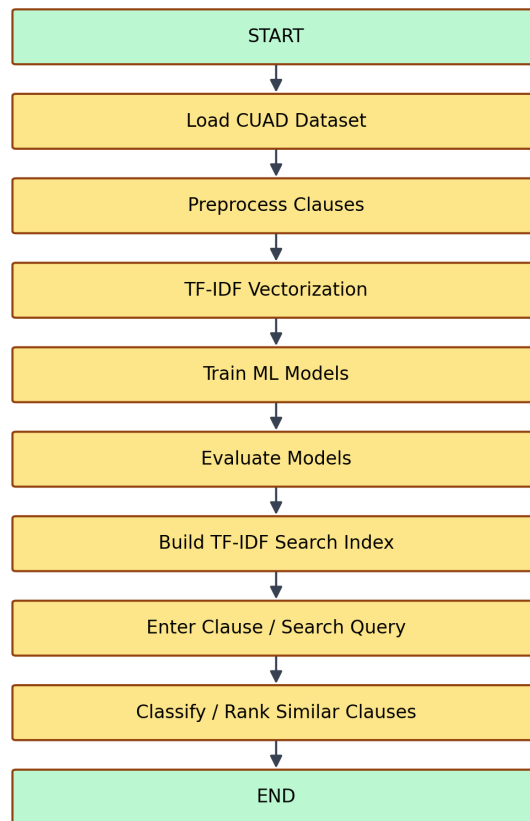


Figure 3.11: Activity diagram: TF-IDF + classical ML clause classification and search pipeline

Step-by-step explanation: the CUAD dataset is loaded and clauses are preprocessed (Section 3.2.1); TF-IDF vectorisation converts clause text into numerical feature vectors; the four classical ML models are trained on these vectors; each model is evaluated using the metrics identified in Section 3.2.3; a TF-IDF search index is built from the same vectors; when a user enters a clause or search query, it is vectorised in the same feature space and either classified by the trained model or ranked against the corpus by cosine similarity, and the results are returned.

3.5. Project Pipeline

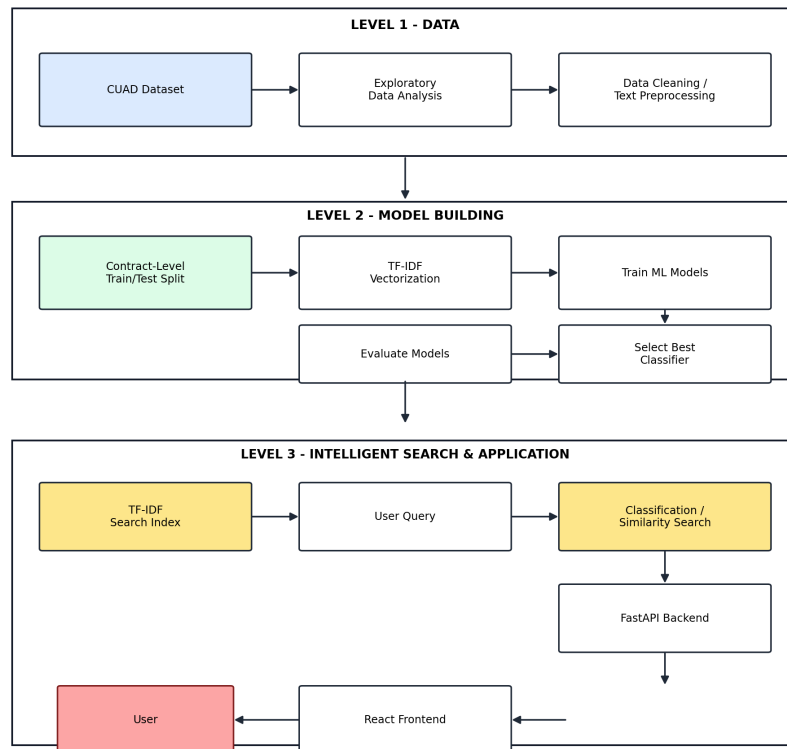


Figure 3.12: ClauseIQ three-level project pipeline

Level 1 (Data): the CUAD dataset is loaded, explored (Section 3.1), and cleaned/preprocessed (Section 3.2). *Level 2 (Model Building):* the cleaned dataset is split at the contract level (so clauses from the same contract do not appear in both training and testing), TF-IDF vectors are computed, the four classical models are trained and evaluated, and the best-performing classifier is selected. *Level 3 (Intelligent Search & Application):* a TF-IDF search index supports cosine-similarity ranking of clauses against a user query; the selected classifier and search index are served through the FastAPI backend and exposed to the user through the React frontend.

3.6. Feasibility Analysis

Technical Feasibility

ClauseIQ is built entirely on mature, freely available technologies: Python and scikit-learn for TF-IDF vectorisation and classical ML; FastAPI for the backend API; React for the frontend; and Supabase PostgreSQL for persistent storage. All of these run on standard computing resources without requiring specialised hardware such as a GPU, making the project technically feasible within a single-semester timeline.

Economical Feasibility

All core software — Python, scikit-learn, pandas, NumPy, FastAPI, React, and Supabase’s free/hobby tier — is open-source or free to use for a project at this scale, and CUAD is a freely licensed public research dataset. No paid annotation, GPU cloud compute, or proprietary licensing is required, keeping the project’s cost close to zero beyond development time.

Operational Feasibility

A user uploads a contract (or enters a search query) through the React frontend; the FastAPI backend extracts and classifies clauses, and the results — category predictions, confidence scores, and similar clauses — are displayed back to the user. This request/response flow does not require specialised training to operate, making the system operationally realistic for its intended users.

3.7. System Environment

3.7.1. Software Environment

- Python 3 — core language for data processing and ML
- FastAPI — backend web framework and REST API layer
- React — frontend user interface framework
- scikit-learn — TF-IDF vectorisation and classical ML classifiers
- pandas, NumPy — data handling and numerical computation
- Supabase (PostgreSQL) — persistent database layer
- Git / GitHub — version control

3.7.2. Hardware Environment

Development and testing require only a standard computer: a dual-core (or better) processor, at least 8 GB of RAM, and approximately 5–10 GB of free storage for the dataset, dependencies, and project files. No GPU or specialised hardware is required, since ClauseIQ uses classical ML rather than deep learning.

3.8. Database Analysis / Supabase Database

ClauseIQ uses Supabase PostgreSQL as its persistent database layer, sitting behind the FastAPI backend:

React Frontend → *FastAPI Backend* → *Supabase PostgreSQL*

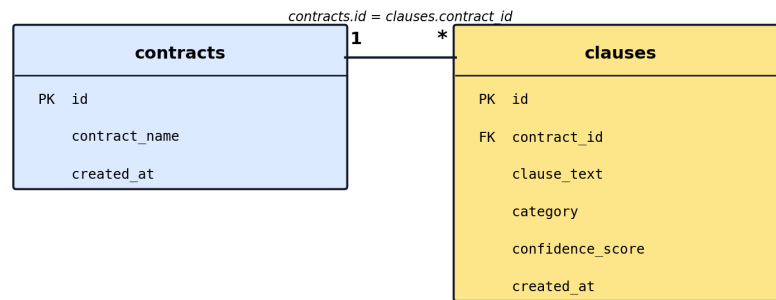
The schema currently consists of two tables, `contracts` and `clauses`, related in a one-to-many relationship (`contracts.id` → `clauses.contract_id`): one contract can contain many clauses.

Field	Description
<code>id</code>	Unique contract identifier (Primary Key)
<code>contract_name</code>	Name of the contract
<code>created_at</code>	Contract creation timestamp

Table 3.1: Table: contracts

Field	Description
id	Unique clause identifier (Primary Key)
contract_id	References contracts.id (Foreign Key)
clause_text	Extracted clause text
category	Predicted clause category
confidence_score	ML classification confidence
created_at	Clause creation timestamp

Table 3.2: Table: clauses



One contract can contain many clauses (One-to-Many Relationship)

Figure 3.13: ClauseIQ database schema: contracts (1) — (many) clauses

The database module is responsible for storing contract metadata, storing extracted clause text, storing the predicted category and confidence score for each clause, recording creation timestamps, and maintaining the foreign-key relationship between contracts and clauses. Data flows from the FastAPI backend to Supabase: after the ML pipeline classifies a contract’s clauses, FastAPI writes one `contracts` row and one `clauses` row per extracted clause. *PDF files themselves are not stored in Supabase Storage, and user authentication and search history are not currently part of the core implementation; only the schema described above exists in the current database.*

4. SYSTEM DESIGN

This chapter presents ClauseIQ’s system design: use case, activity, sequence and class diagrams, and the database design. All diagrams reflect the actual current implementation — a React frontend, a FastAPI backend, a classical ML/TF-IDF processing layer, and a Supabase PostgreSQL database — with no login, search-history, or file-storage features, since these are not part of the current system.

4.1. Use Case Diagram

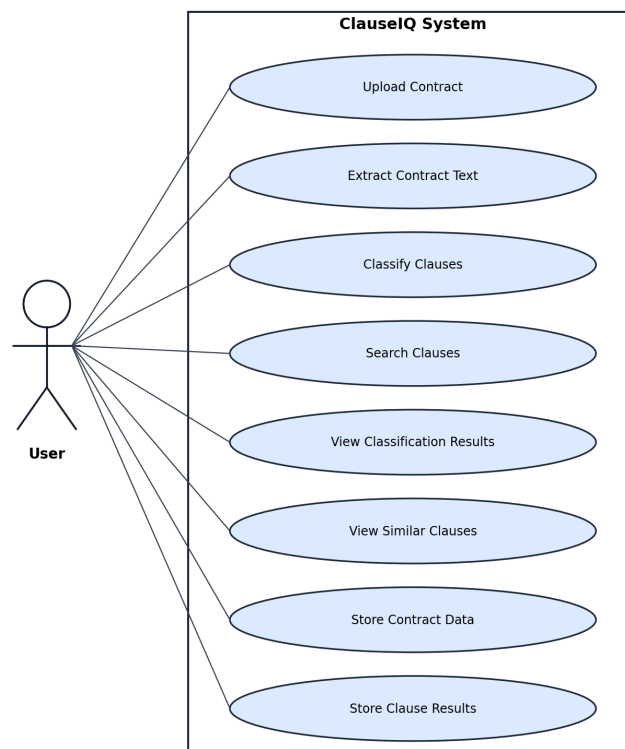


Figure 4.1: ClauseIQ use case diagram

The primary actor is the *User*. The system supports eight use cases: Upload Contract, Extract Contract Text, Classify Clauses, Search Clauses, View Classification Results, View Similar Clauses, Store Contract Data, and Store Clause Results. No login or registration use case is included, since user authentication is not part of the current implementation.

4.2. Activity Diagram

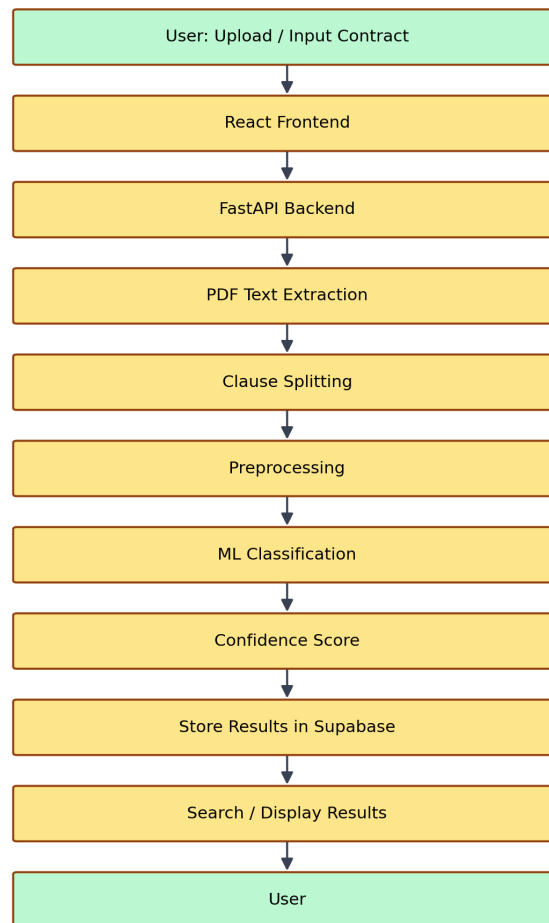


Figure 4.2: End-to-end application activity diagram

The activity diagram traces a single user interaction end to end: the user uploads or inputs a contract through the React frontend, which forwards the request to FastAPI; FastAPI performs PDF text extraction, clause splitting and preprocessing; the ML classification module assigns a category and confidence score to each clause; results are stored in Supabase; and the search/display step returns results to the user. This is distinct from the algorithm-level activity diagram in Section 3.4.1, which describes the model-training pipeline rather than a live user request.

4.3. Sequence Diagram

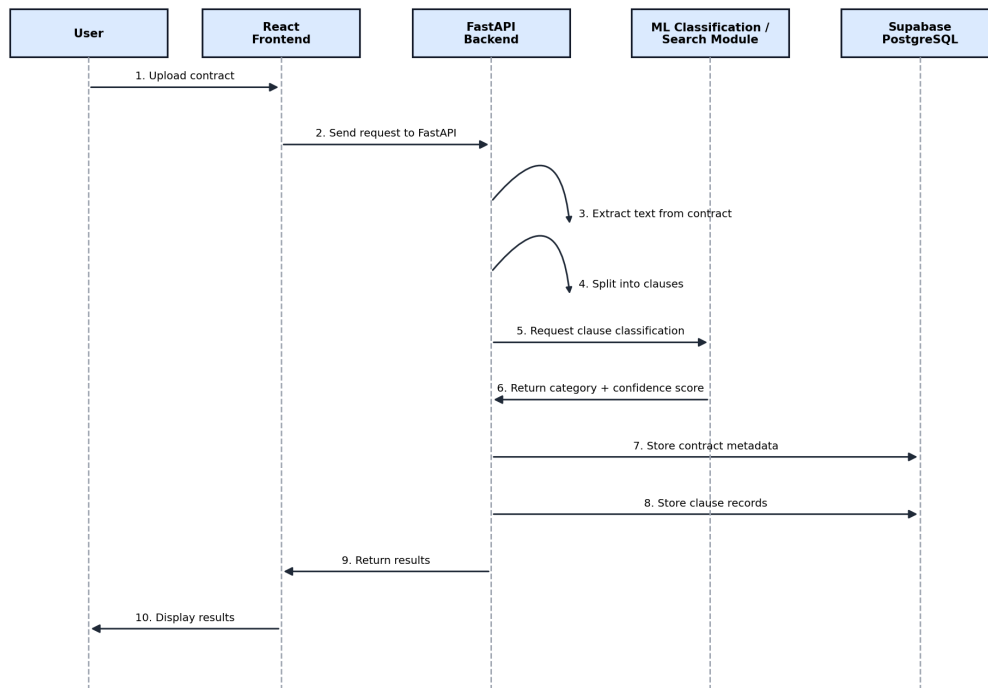


Figure 4.3: Sequence diagram for the contract-upload and classification flow

The sequence diagram shows five participants — User, React Frontend, FastAPI Backend, ML Classification/Search Module, and Supabase PostgreSQL — and the ten-step message flow: the user uploads a contract (1); React forwards the request to FastAPI (2); FastAPI extracts text (3) and splits it into clauses (4); FastAPI requests classification from the ML module (5), which returns a category and confidence score (6); FastAPI stores the contract metadata (7) and clause records (8) in Supabase; and results are returned to React (9) and displayed to the user (10). The same participants support an analogous search flow, in which a query is vectorised and compared against the TF-IDF search index (Section 3.4.1) instead of steps 3–6.

4.4. Class Identification

Class	Attributes	Responsibilities
Contract	id, contract_name, created_at	Represents one uploaded contract; exposes its associated clauses
Clause	id, contract_id, clause_text, category, confidence_score	Represents one classified clause and its predicted category
TFIDFProcessor	vocabulary, idf_weights	Fits and applies TF-IDF vectorisation to clause text
MLClassifier	model_type, trained_model	Trains a classical ML model and predicts clause categories
SearchEngine	tfidf_index	Ranks clauses by cosine similarity to a query
FastAPIService	routes	Exposes classification and search endpoints to the frontend
SupabaseRepository	connection	Persists and retrieves contract and clause records

Table 4.1: Identified classes, attributes and responsibilities

Contract and Clause correspond directly to the contracts and clauses database tables (Section 3.8); TFIDFProcessor, MLClassifier and SearchEngine implement the ML/NLP processing layer; FastAPIService and SupabaseRepository represent the backend API and database-access layers respectively.

4.5. Class Diagram

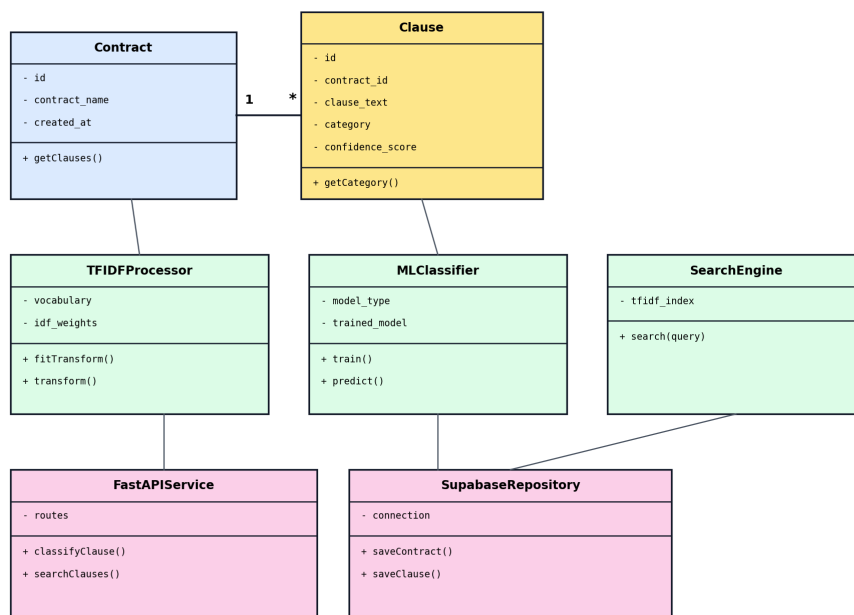
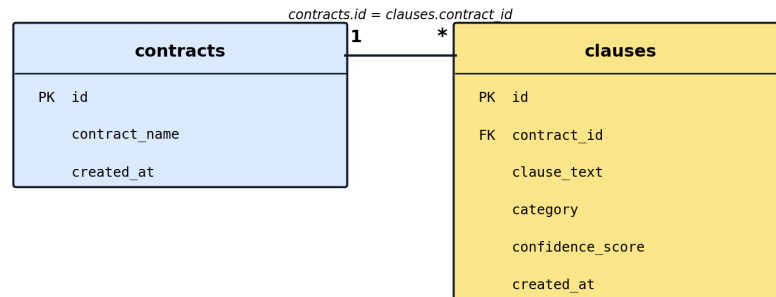


Figure 4.4: ClauseIQ class diagram

The key relationship is between **Contract** and **Clause**: one **Contract** is associated with many **Clause** instances (1-to-*), mirroring the database's one-to-many relationship. **TFIDFProcessor**

and `MLClassifier` operate on Clause text; `FastAPIService` calls into the processing classes; and `SupabaseRepository` persists Contract and Clause data.

4.6. Database Design



One contract can contain many clauses (One-to-Many Relationship)

Figure 4.5: Entity-relationship design for the Supabase database

The database consists of exactly two tables. `contracts` stores contract-level metadata (`id` as primary key, `contract_name`, `created_at`). `clauses` stores clause-level records (`id` as primary key; `contract_id` as a foreign key referencing `contracts.id`; `clause_text`; `category`; `confidence_score`; `created_at`). The relationship `contracts.id → clauses.contract_id` is one-to-many: a single contract can have many associated clause records, but each clause belongs to exactly one contract. No additional tables, columns, authentication fields, or storage-path fields exist in the current schema.